COMPLETE SYSTEM GUIDE

POUND × Flatterie
Online Booking System

From the screen to the database: how every part works, and how to explain it

> **Who this is for:** the member teaching the group, as a reference for explaining the whole system.
>
> **How to read it:** Part A gives the big picture. Parts B and C tour the backend and frontend. Part D follows every feature step by step (screen → request → route → checks → logic → database → screen). Parts E–G cover data, security and running the system. Part H is a teaching plan with practice questions.

1. **A.** The big picture
2. **B.** Backend tour (server)
3. **C.** Frontend tour (client)
4. **D.** Feature walkthroughs (14 features)
5. **E.** Data and the seed script
6. **F.** Security summary
7. **G.** Running the system and common errors
8. **H.** Teaching plan and practice questions

A. The Big Picture

A1. Three separate parts

Part	Technology	Job
Frontend (client)	React + Vite, React Router, GSAP (animations)	Shows pages, collects input, sends requests. Runs in the browser at `localhost:5173`.
Backend (server)	Node.js + Express, Mongoose, bcryptjs, jsonwebtoken	Receives requests, checks login and role, applies the business rules, reads and writes the database. Runs at `localhost:5000`.
Database	MongoDB Atlas (cloud)	Stores everything permanently: `users`, `tables`, `reservations`.

The frontend and backend are **two separate programs** that talk using **HTTP requests with JSON**. That's what makes it a **REST API**: each URL is a resource (`/api/reservations`) and the method says what to do (GET = read, POST = create, PUT/PATCH = change).

A2. The path of every request



> **Explain it like this:** "The page never touches the database. It asks the server through the API. The route sends the request to the right controller, the middleware checks who you are first, and only the controller talks to the database through the models."

A3. Every answer has the same shape

```
{ "success": true,  "message": "Reservation submitted...", "data": { ... } }
{ "success": true,  "count": 12, "data": [ ... ] }
{ "success": false, "message": "Validation failed", "errors": ["guests must be ..."] }
```

Status codes used: **200** OK · **201** created · **400** invalid input or rule broken · **401** not logged in or bad token · **403** logged in but not allowed · **404** not found · **409** conflict (e.g. table already booked, email already used) · **500** unexpected error.

B. Backend Tour (server folder)

B1. `server.js` : where the server starts

1. Loads settings from `.env` (`PORT`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `CLIENT_URL`, `MONGO_URI`) and **stops** if `JWT_SECRET` or `MONGO_URI` is missing.
2. Adds middleware for every request: `cors()` (lets the website on port 5173 call port 5000) and `express.json()` (reads JSON bodies).
3. `GET /api/health` answers with the server and database status.
4. Connects the route files: `/api/auth`, `/api/users`, `/api/tables`, `/api/reservations`, `/api/customers`, `/api/archive`.
5. Adds `notFound` (unknown URL → 404) and `errorHandler` (any error → clean JSON) **after** all routes.
6. **Connects to MongoDB first** (`connectDB()` in `config/db.js`), and only then starts listening.

B2. `config/`

File	Contents
`businessRules.js`	The restaurant rules in one place: `TIME_SLOTS` (11:00, 13:00, 15:00, 17:00, 19:00), `MIN_GUESTS` 1, `MAX_GUESTS` 10, `BOOKING_WINDOW_DAYS` 30, the 4 statuses, `ACTIVE_STATUSES` (Pending, Confirmed: these hold a table), `ALLOWED_STATUS_CHANGES` (the workflow).
`db.js`	`connectDB()` connects with Mongoose; prints an error and stops if it fails. `getDatabaseStatus()` for the health check.

B3. `models/` : the shape of the data

Model	Fields	Rules
User	fullName, email, phone, passwordHash, role, status, resetTokenHash, resetTokenExpires, archived, archivedAt, archivedBy	email unique + lowercase; role admin/staff/customer; status active/inactive; `toJSON` removes the password hash and reset token whenever a user is sent out
Table	tableNumber, capacity, location, status	tableNumber unique; capacity 1–20; status available ("In Service") / unavailable ("Out of Service")
Reservation	customer (→ User or null), customerName, customerEmail, customerPhone, table (→ Table), date, time, guests, specialRequest, status, cancelledBy, cancellationReason, archived, archivedAt, archivedBy	time must be one of the 5 slots; guests 1–10; status one of 4; **index on table + date + time** (fast conflict check)

All models use `timestamps` (automatic `createdAt` / `updatedAt`). A reservation stores only the **IDs** of its customer and table (a **reference**); `populate("table")` fills in the full table when the data is sent to the website. **date** is text `"YYYY-MM-DD"` and **time** is `"HH:MM"`, which compare correctly as plain text.

B4. `middleware/` : the checks before the logic

Function	What it does
`protect`	Reads `Authorization: Bearer <token>` → verifies the token's signature and expiry with `JWT_SECRET` → loads the user from the database → refuses (401) if missing or **inactive** → saves the user in `req.user`.
`allowRoles(...roles)`	Refuses (403) if `req.user.role` isn't in the list, e.g. `allowRoles("admin", "staff")`.
`errorHandler`	Turns errors into JSON: invalid ID → 400, schema rule broken → 400, duplicate unique value → 409, anything else → 500.
`notFound`	Unknown URL → 404 "Route not found".

B5. All API routes

Method	URL	Who	Controller function
POST	/api/auth/register	Anyone	register
POST	/api/auth/login	Anyone	login
POST	/api/auth/forgot-password	Anyone	forgotPassword
GET	/api/auth/reset-password/:token	Anyone	checkResetToken
POST	/api/auth/reset-password	Anyone	resetPassword
GET / PUT	/api/auth/me	Logged in	getMe / updateMe
PUT	/api/auth/me/password	Logged in	changePassword
GET, POST	/api/users , /api/users/:id	Admin	getAllUsers, getUserById, createUser
PUT	/api/users/:id	Admin	updateUser
GET	/api/tables , /api/tables/:id	Anyone	getAllTables, getTableById
POST / PUT	/api/tables , /api/tables/:id	Staff, Admin	createTable / updateTable
GET	/api/reservations/availability	Anyone	getAvailableTables
GET	/api/reservations/slots	Anyone	getSlotAvailability
GET	/api/reservations/my	Customer	getMyReservations
GET	/api/reservations	Staff, Admin	getAllReservations
GET	/api/reservations/:id	Logged in (customer: own only)	getReservationById
POST	/api/reservations	Logged in	createReservation
PUT	/api/reservations/:id	Staff, Admin	updateReservation
PATCH	/api/reservations/:id/status	Staff, Admin	updateReservationStatus
PATCH	/api/reservations/:id/cancel	Customer (own)	cancelReservation
GET	/api/customers , /api/customers/:id	Staff, Admin	getCustomers, getCustomerById

GET / PATCH	/api/archive/reservations (.../:id/archive, .../:id/restore)	Staff, Admin	getArchivedReservations, archiveReservation, restoreReservation
GET / PATCH	/api/archive/users (.../:id/archive, .../:id/restore)	Admin	getArchivedUsers, archiveUser, restoreUser

Order matters in `reservationRoutes.js`: fixed paths (`/availability`, `/slots`, `/my`) come before `/:id`, otherwise "my" would be read as an ID.

B6. `utils/`

File	Contents
`authUtils.js`	`generateToken(user)` creates the JWT (user id + role, signed, expires per `JWT_EXPIRES_IN`); `toPublicUser(user)` returns the user without private fields.
`validators.js`	Shared rules: email pattern, PH mobile pattern (`09...` or `+639...`, 11/13 digits), password rule (8+ characters, a letter and a number), `escapeRegex` (makes search text safe).
`dateUtils.js`	`getToday()`, `addDays()`, `getCurrentTime()`, `isValidDateString()`.
`emailService.js`	`sendPasswordResetEmail()`: simulated, prints the email with the link in the server terminal.

C. Frontend Tour (client folder)

C1. Start-up

`main.jsx` starts React inside `<BrowserRouter>` (page URLs) and `<AuthProvider>` (who is logged in, shared by all pages). `App.jsx` is the list of pages and their URLs.

C2. Pages and who can open them (`App.jsx`)

URL	Page	Access
`/`	HomePage	Anyone
`/login`, `/register`, `/forgot-password`, `/reset-password/:token`	Account pages (inside AuthLayout)	Anyone
`/my-reservations`	MyReservationsPage	Customers
`/profile`	ProfilePage	Any logged-in user
`/dashboard` (+ reservations, tables, customers, archive)	Dashboard pages (inside DashboardLayout)	Staff, Admin
`/dashboard/users`	ManageUsersPage	Admin only
anything else	NotFoundPage	Anyone

`ProtectedRoute` guards pages: not logged in → sent to `/login`; wrong role → sent away (home, or `/dashboard` for staff trying to open Users). This only **hides** pages. The real protection is on the server (`protect` + `allowRoles`).

C3. Layouts

`PublicLayout`	Navbar + page + Footer, for the website pages.
`AuthLayout`	Food photo with the logo on one side, the form on the other (Login, Register, Forgot/Reset). Has a short entrance animation.
`DashboardLayout`	Sidebar + page, for staff/admin. The sidebar hides "Users" from staff.

C4. Who is logged in: `context/AuthContext.jsx`

- Keeps `user` in memory and the **token in `localStorage`** (the browser's storage), so a refresh keeps you logged in.
- When the site opens with a saved token, it calls `GET /api/auth/me` to learn who you are. If the server says 401 (expired, fake, or deactivated), the token is removed.
- Gives every page `useAuth()` → `user`, `login`, `register`, `logout`, `updateUser`.

C5. Talking to the server: `api/`

`apiClient.js` has `apiRequest()`, used by every API file. It builds the URL (`http://localhost:5000/api…`), sends JSON, **adds the token automatically** (`Authorization: Bearer …`), and turns a `{ success: false }` answer into an error with the server's message, which the page then shows. The other files group functions by area: `authApi`, `userApi`, `tableApi`, `reservationApi`, `customerApi`, `archiveApi`.

C6. Reusable components

Component	Used for
Navbar	Logo, section links (highlights the section you're on), ☰ menu on phones, profile menu; transparent over the hero photo
BookingForm	The Book a Table form on the home page
ReservationForm	Staff pop-up for new phone/walk-in bookings and edits
UserForm / TableForm	Admin add/edit user; staff add/edit table (one form, two modes)
Modal	The pop-up window used by all the forms
PasswordInput	Password box with the eye button
MenuCarousel, MenuItemDetails, Photo	Moving menu cards, dish details pop-up, photo with placeholder
OpenStatusBadge	"Open now / Closed" using Philippine time
ProtectedRoute, Sidebar, Footer	Page guard, dashboard menu, footer with disclaimer

Hooks (reusable logic): `useHomeAnimations` (GSAP: hero entrance, scroll reveal, parallax), `useAuthAnimations` (account pages entrance), `useActiveSection` (which home section is on screen). **Utils**: `dateHelpers` (format dates/times), `formatRef` (short ref `#EA7886`), `formatPrice`, `images`.

D. Feature Walkthroughs

Each feature is traced from the screen to the database and back.

D1. Register

Screen	RegisterPage: name, email, phone, password, confirm password (passwords must match on the page)
Request	`POST /api/auth/register` (no login needed)
Logic	`register`: validates all fields → checks the email isn't used (409) → hashes the password with bcrypt → creates the user with role **always "customer"** (even if someone sends "admin")
Answer	201 + a token + the user → the website saves the token, so the new customer is logged in immediately

D2. Login and staying logged in

Screen	LoginPage → `login()` from AuthContext
Request	`POST /api/auth/login`
Logic	Finds the user by email → compares the password with the hash (`bcrypt.compare`) → wrong email **or** wrong password gives the **same** message (attackers can't tell which emails exist) → inactive account → 403
Answer	Token (JWT) + user. The token is saved in localStorage and sent with every later request.

Logout just removes the token from the browser. Without it, protected requests get 401.

D3. Profile and change password

Screen	ProfilePage: details form + change password form
Requests	`PUT /api/auth/me` (name, email, phone) · `PUT /api/auth/me/password`
Logic	Profile: validates; a new email can't belong to another account (409); role and status can't be changed here. Password: the **current password is required** and must be correct; the new one must follow the rule and be different; it's saved as a new hash.

D4. Forgot / reset password

Step 1	ForgotPasswordPage → `POST /api/auth/forgot-password` with the email
Logic	If the account exists and is active: creates a random token (64 characters), stores only its **SHA-256 hash** + an expiry **15 minutes** ahead, and "emails" the link (printed in the server terminal). The answer is the **same** whether the email exists or not.
Step 2	Opening the link → ResetPasswordPage first calls `GET /api/auth/reset-password/:token` to check it's still valid
Step 3	`POST /api/auth/reset-password` with the token + new password → saves the new hash and **clears the token** (one-time use)

Explain it like this: "We store only a hash of the reset token, like passwords. Even someone who sees the database can't use it. It expires in 15 minutes and works once."

D5. Customer books a table (core)

Screen	BookingForm (home page). Must be logged in as a customer; contact details are pre-filled from the profile.
Slots	When date or guests change: `GET /api/reservations/slots?date=&guests=` → for each of the 5 seatings: available or not, and how many tables are left. Buttons show "Available", "Only 2 tables left", "Last table!", "Fully booked" or "Time has passed".
Request	`POST /api/reservations` (protect)
Logic	`createReservation`: validates (name, email, PH phone, date not past and ≤ 30 days ahead, time is a slot and not passed, guests 1–10) → a customer can't hold 2 active bookings in the same slot (409) → **any table the customer sends is ignored** → **auto-assign**: `findAvailableTables()` returns in-service tables big enough and not booked in that slot, **smallest first**; the first one is used → none free → 409
Database	Saved with status **Pending** and `customer` = the account's ID
Answer	201 + the booking with its table details → success message with the table number

How `findAvailableTables()` works (the conflict check)

1. `Reservation.distinct("table", { date, time, status in [Pending, Confirmed] })` → the IDs of tables already taken in that slot.
2. `Table.find({ status: "available", capacity ≥ guests, _id not in that list })`, sorted by capacity then table number.

Explain it like this: "First we ask which tables are taken in that seating. Then we look for in-service tables that are big enough and not taken, smallest first. Cancelled and completed bookings don't block a table, only Pending and Confirmed do."

D6. My Reservations and cancelling

Screen	MyReservationsPage: Upcoming tab (active, time not reached) and History tab (everything else). Shows status, table, short ref, and the cancellation reason if any.
Requests	`GET /api/reservations/my` · `PATCH /api/reservations/:id/cancel` with an optional reason
Logic	Cancel only **your own** booking (403 otherwise), only Pending/Confirmed, only **before it starts**. Status → Cancelled, `cancelledBy: "customer"`. The table is free again automatically, because the check only counts active bookings.

D7. Staff: status workflow

Screen	ManageReservationsPage: filters (Upcoming/Today/Past/All, status, search by name/email/phone/ref). The buttons shown depend on status and time.
Request	`PATCH /api/reservations/:id/status` with `{ status, reason }`
Logic	`updateReservationStatus` checks `ALLOWED_STATUS_CHANGES`: Pending → Confirmed/Cancelled; Confirmed → Completed/Cancelled; Completed and Cancelled → nothing. Time rules: **Confirm only before** the seating starts; **Complete only after** it starts. Cancelling saves `cancelledBy: "staff"` + the reason (Reject, Cancel and No-show all use this).

Status / time	Buttons
Pending, before start	Confirm · Edit · Reject
Confirmed, before start	Edit · Cancel
Confirmed, after start	Complete · No-show
Completed or Cancelled	Archive

D8. Staff: edit and phone/walk-in bookings

Screen	ReservationForm in a pop-up. The Table list shows **only free tables** (from `GET /api/reservations/availability`) and reloads when date, time or guests change.
New	`POST /api/reservations` as staff: the table can be chosen (checked: exists 404, in service 400, big enough 400, not booked 409) or auto-assigned; `customer: null` (no account)
Edit	`PUT /api/reservations/:id`: only Pending/Confirmed that haven't started; status can't be changed here; current data + changes are merged and validated with the same rules; a table must be chosen
Key idea	When editing, the booking would "clash with itself". The check **excludes its own ID** (`excludeId` / `?exclude=`).

D9. Tables and daily schedule

ManageTablesPage has 2 tabs. **Tables:** add/edit (`POST` / `PUT /api/tables`; number unique, capacity 1–20), take a table **out of service** (status unavailable, with a warning if it still has upcoming bookings). Tables are **never deleted**, so old bookings keep their table. **Daily Schedule:** a grid of tables × 5 seatings for a chosen date, built from that day's reservations (Pending, Confirmed, Completed shown; cancelled ignored).

D10. Customers page

`GET /api/customers` (staff/admin, view only): customer accounts that aren't archived, with search (name, email, phone) and status filter. For each, a summary from their reservations: total, completed, cancelled, upcoming, **last visit** (latest completed date). `GET /api/customers/:id` adds the full history. It's separate from `/api/users` on purpose: staff can **see** customers but not **manage** accounts.

D11. User management (admin)

Screen	ManageUsersPage: list with search and role/status filters, Add User, Edit, Activate/Deactivate, Archive (inactive only)
Requests	`GET/POST /api/users`, `PUT /api/users/:id` (all admin-only through `router.use(protect, allowRoles("admin"))`)
Rules	Any role can be created (this is how staff accounts are made). Passwords can't be changed here. The admin **can't change their own role** or **deactivate themselves**. There must **always be at least one active admin**. Archived accounts must be restored before editing. A deactivated user is blocked at login **and** on every request (protect checks status).

D12. Archive

Bookings	`PATCH /api/archive/reservations/:id/archive` (staff/admin): only **Completed or Cancelled**. Sets `archived: true`, `archivedAt`, `archivedBy`. The main list hides archived bookings; the customer still sees them in their history.
Accounts	`PATCH /api/archive/users/:id/archive` (admin): only **inactive** accounts, so an archived account can never log in. Hidden from Users and Customers.
Archive page	Search (name, email, phone, or ref like `#EA7886`), shows who archived and when, **Restore** (booking keeps its status; account comes back inactive). Staff see only the bookings tab; admin also sees accounts.

> **Explain it like this:** "Archiving is a flag, not a delete. The record stays in the database, separated from active records, and can be restored. That's called a soft delete."

D13. Overview (dashboard home)

No special backend route. The page loads 3 lists at the same time with `Promise.all`: today's reservations, Pending reservations, and tables. It then counts bookings today, guests expected, pending requests not yet passed, and tables in service. The numbers are always current because they're calculated each time, not stored.

D14. Home page extras

- **Menu carousel:** the dishes are placed twice in a row and a CSS animation slides the row by exactly one copy, then restarts, so it looks endless. Hover pauses it; click opens details.
- **Navbar:** transparent over the hero photo, brown after; underlines the section you're on (the most visible one); ☰ menu on phones.
- **Visit Us:** embedded Google Map, Get Directions link, tap-to-call (`tel:`) and email (`mailto:`) links, **Open now / Closed** badge (10 AM–10 PM, Philippine time, rechecked every minute).
- **Animations (GSAP):** hero entrance, sections sliding in on scroll, parallax photo. Turned off for users who disabled animations on their device.

E. Data and the Seed Script

`npm run seed` runs `server/seed.js`, which loads the demo data in `server/data/`:

Data	Contents
Tables (12)	Table 1–4: 2 seats · 5–8: 4 seats · 9–10: 6 seats · 11: 10 seats, out of service · 12: 10 seats, Function Room
Users (12)	Admin: Giuz Montaño · Staff: Marco Delos Reyes, Rene Baterbonia · 9 customers (Paolo Garcia is inactive)
Reservations (40)	**26 Completed** (2 archived), 5 Cancelled (1 archived), 1 Confirmed from yesterday, 2 today, 6 upcoming. Mix of account and walk-in bookings.

1. **Checks the data first** (`checkSampleData()`) with the system's rules: tables exist, the group fits the table, no table used twice in a slot, no upcoming booking on an out-of-service table, a customer can't hold 2 active bookings in one slot. Any problem → it stops, and nothing is changed.
2. Deletes the 3 collections.
3. Inserts tables, then users, then reservations, converting the sample IDs (`"t5"`, `"u3"`) into the new MongoDB IDs so the references stay correct.
4. Prints a summary (users by role, reservations by status).

> **Dates are counted from the day you run the seed** (`day(-7)` = 7 days ago). Run it on presentation morning so "today" and "upcoming" bookings match that day. It also does the required **data cleanup**. The database is shared, so tell the group first.

F. Security Summary

Protection	How
Passwords	bcrypt hash (10 salt rounds); never stored or sent as typed; `toJSON` removes the hash from every answer
Login sessions	Signed JWT with user id + role, expires (default 1 day); verified on every protected request
Permissions	Checked on the **server** (`protect`, `allowRoles`, ownership checks like "your own booking only"), not just hidden on the website
Deactivated accounts	Blocked at login and on every request, even with an old token
Reset links	Random, stored as SHA-256 hash, 15-minute expiry, one-time use; same answer whether the email exists or not
Input	Validated on the website and again on the server; models add a second layer (enum, min/max, unique)
Secrets	`JWT_SECRET` and `MONGO_URI` live in `.env`, which is excluded from GitHub
Search	`escapeRegex()` makes typed text safe before searching

G. Running the System and Common Errors

1. Install once: `npm install` in `server` and in `client`.
2. `server/.env` needs: `PORT`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `CLIENT_URL`, `MONGO_URI` (see `.env.example`).
3. Terminal 1: `cd server` → `npm run dev` (nodemon restarts it when a file is saved; type `rs` to restart manually).
4. Terminal 2: `cd client` → `npm run dev` → `http://localhost:5173`.

Message	Cause → fix
"Cannot connect to the server. Is the backend running?"	Backend not started → run it in terminal 1
`bad auth : authentication failed`	Wrong password in `MONGO_URI` → fix `.env`
Timeout / `querySrv ENOTFOUND`	No internet or Wi-Fi blocks MongoDB → use a hotspot
`MONGO_URI is missing`	Line not in `server/.env` or not saved
"Failed to resolve import …"	A file name or folder is wrong, or `npm install` wasn't run (e.g. for GSAP)
Logged out unexpectedly	Token expired, account deactivated, `JWT_SECRET` changed, or the database was re-seeded → log in again

H. Teaching Plan and Practice Questions

Suggested order for teaching the group

1. **Part A** with the restaurant picture: website → waiter (routes) → guard (middleware) → kitchen (controllers) → storage (MongoDB).
2. **Run the system together** (Part G) and log in with each demo account.
3. **D5 Booking** first: it touches every layer and contains the main rule (conflict check).
4. **D7–D8 Staff workflow**, then **D12 Archive**.
5. **D1–D4 Accounts** and **D11 User management**, then **Part F Security**.
6. Each member reads their presentation part's sections again and answers the questions below.

What each member must be able to explain

Member	Sections	Practice questions
Al-Vi	A, E, the requirements	What does MERN stand for and what does each part do? · How did you create the 20 completed transactions? · What does the seed check before loading? · What is out of scope?
Giuz	A, B, C, F	Walk through one request from page to database. · What do `protect` and `allowRoles` do? · Why is the token saved in localStorage? · What is a reference and what does `populate` do? · Why the index on table + date + time?
Kclei	D1–D4, D11	Why does login give the same message for a wrong email and a wrong password? · How is the reset link kept safe? · Can a registered user become admin? · Which safety rules protect the admin account?
Khalil	D5–D10, D12, D13	How does the system pick a table? · Which statuses block a table? · Why can't you confirm a booking that already started? · Why does editing exclude its own ID? · What can be archived and why not delete?

Answers to check yourselves

Question	Answer
Why is the token in localStorage?	So the user stays logged in after a refresh. On start, AuthContext sends it to `/api/auth/me` to confirm it's still valid.
Why the same login error message?	So attackers can't find out which emails have accounts.
Can a registered user become admin?	No. `register` always sets role "customer". Only an admin can create staff or admin accounts.
Which statuses block a table?	Only Pending and Confirmed (`ACTIVE_STATUSES`). Cancelled frees the table; Completed is in the past.
Why can't you confirm a started booking?	Confirming means "we'll hold your table"; once the seating started, staff should Complete it or mark it a no-show instead.
Why does editing exclude its own ID?	Otherwise the booking would see its own table as taken and refuse to save.

Why a separate `/api/customers`?	`/api/users` is admin-only and can edit accounts; staff only need to view customers (least privilege).
What does `populate` do?	Replaces a stored ID (e.g. the table's ID) with the full record when sending data to the website.
Why is the Overview always correct?	Its numbers are calculated from the live data every time the page opens, not stored.

All statements in this guide describe the code as of the latest version in the repository.